

Module 4: Online Evidence And Production Feedback Loop

In this module, you will turn deployed-agent behavior into evidence that can improve the evaluation dataset.

The production feedback loop has four moving parts:

1. Observe real agent sessions through online evaluation and traces.
2. Collect the sessions, tool calls, and safe metadata needed to understand what happened.
3. Mine those sessions for examples worth reviewing.
4. Add reviewed examples to the managed evaluation dataset as a new immutable version.

The main concept to watch for is the difference between a trace, a candidate, and a dataset example. A trace is what happened. A candidate is a trace that looks useful. A dataset example is a candidate that has been reviewed, enriched with expectations, and published into AgentCore Evaluation.

Step 1: Load The Deployment And Dataset Context

This cell reconstructs the working context from saved manifest files instead of relying on notebook memory.

You should learn how later lifecycle steps find the deployed runtime, the managed dataset, the baseline dataset version, the CloudWatch log groups, and the service names needed to query traces. The output should give you the identifiers you will use for the rest of the notebook.

```
In [1]: import json
import os
import re
import sys
import time
import uuid
from datetime import datetime, timezone, timedelta
from pathlib import Path

import boto3
import pandas as pd
from botocore.exceptions import ClientError

SECTION_DIR = Path.cwd().resolve()
if SECTION_DIR.name != "04-online-eval-observability":
    SECTION_DIR = Path("04-online-eval-observability").resolve()
REPO_ROOT = SECTION_DIR.parent
SECTION03_DIR = REPO_ROOT / "03-production-deployment"
os.chdir(SECTION_DIR)

for path in [SECTION_DIR, SECTION03_DIR]:
    if str(path) not in sys.path:
        sys.path.insert(0, str(path))

from evidence_contract import (
    DATASET_UPDATE_MANIFEST_PATH,
    ONLINE_EVIDENCE_MANIFEST_PATH,
```

```

    PRODUCTION_FEEDBACK_CANDIDATES_PATH,
    PROMOTED_FEEDBACK_EXAMPLES_PATH,
    build_dataset_update_manifest,
    build_feedback_candidates,
    build_online_evidence_manifest,
    demo_user_suffix,
    extract_tool_calls_from_spans,
    load_section03_context,
    runtime_log_groups,
    save_json,
    service_names,
    promote_feedback_candidates,
)
from utils import get_user_token, invoke_agent_runtime, redact_email, sanitize_error

NOTEBOOK_STARTED_AT = time.time()
context = load_section03_context()
DEPLOYMENT_MANIFEST = context["deployment"]
DATASET_MANIFEST = context["dataset"]
BATCH_EVALUATION_MANIFEST = context["batch_evaluation"]

REGION = DEPLOYMENT_MANIFEST["region"]
ACCOUNT_ID = DEPLOYMENT_MANIFEST["account_id"]
RUNTIME_ID = DEPLOYMENT_MANIFEST["runtime"]["runtime_id"]
RUNTIME_ARN = DEPLOYMENT_MANIFEST["runtime"]["runtime_arn"]
OTEL_SERVICE_NAME = DEPLOYMENT_MANIFEST["otel_service_name"]
LOG_GROUPS = runtime_log_groups(DEPLOYMENT_MANIFEST)
SERVICE_NAMES = service_names(DEPLOYMENT_MANIFEST)
DATASET_ID = DATASET_MANIFEST["managed_datasets"]["predefined"]["dataset_id"]
BASELINE_DATASET_VERSION = DATASET_MANIFEST["managed_datasets"]["predefined"]["baseline_dataset_version"]

print("Loaded Section 03 context")
print(f"   Region: {REGION}")
print(f"   Runtime: {RUNTIME_ID}")
print(f"   Deployment ID: {DEPLOYMENT_MANIFEST['deployment_id']}")
print(f"   Dataset: {DATASET_ID} version {BASELINE_DATASET_VERSION}")
print(f"   Batch baseline: {BATCH_EVALUATION_MANIFEST['batch_evaluation']['batch_evaluation_id']}")
print("   Log groups:")
for group in LOG_GROUPS:
    print(f"       - {group}")
print(f"   Service names: {SERVICE_NAMES}")

sts_client = boto3.client("sts", region_name=REGION)
iam_client = boto3.client("iam", region_name=REGION)
logs_client = boto3.client("logs", region_name=REGION)
cloudwatch_client = boto3.client("cloudwatch", region_name=REGION)
agentcore_runtime_client = boto3.client("bedrock-agentcore", region_name=REGION)
agentcore_control_client = boto3.client("bedrock-agentcore-control", region_name=REGION)
cognito_client = boto3.client("cognito-idp", region_name=REGION)
assert sts_client.get_caller_identity()["Account"] == ACCOUNT_ID

```

```
Loaded Section 03 context
Region: us-east-1
Runtime: ecommerce_workshop_product_catalog_agent-518f2MA0eC
Deployment ID: section03-20260920T083438Z-51ad2cf5
Dataset: ecommerce_release_gate_5285a57240d1-ZzqKJ5C2HG version 1
Batch baseline: Section03Batch51ad2cf56b6b18Retrydabc9c-6ab901c2d7
Log groups:
- aws/spans
- /aws/bedrock-agentcore/runtimes/ecommerce_workshop_product_catalog_agent-518f2MA0eC-DEFAULT
- /aws/vendedlogs/bedrock-agentcore/ecommerce_workshop_product_catalog_agent-518f2MA0eC
Service names: ['ecommerce_workshop_product_catalog_agent.DEFAULT']
```

Step 2: Create Or Reuse The Evaluation Role

Online evaluation needs an AWS role that can read the trace/log evidence and write evaluation results.

This cell creates or updates that role so the rest of the notebook can configure AgentCore online evaluation. The thing to look for in the output is the role ARN; that ARN becomes part of the online evaluation configuration in the next step.

```
In [2]: EVALUATION_ROLE_NAME = "ecommerce-workshop-evaluation-role"

def create_or_update_evaluation_role():
    trust_policy = {
        "Version": "2012-10-17",
        "Statement": [{
            "Effect": "Allow",
            "Principal": {"Service": "bedrock-agentcore.amazonaws.com"},
            "Action": "sts:AssumeRole",
            "Condition": {"StringEquals": {"aws:SourceAccount": ACCOUNT_ID}},
        }],
    }
    permissions_policy = {
        "Version": "2012-10-17",
        "Statement": [
            {
                "Sid": "BedrockModelAccess",
                "Effect": "Allow",
                "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
                "Resource": [
                    f"arn:aws:bedrock:{REGION}::foundation-model/*",
                    f"arn:aws:bedrock:*:{ACCOUNT_ID}:inference-profile/*",
                ],
            },
            {
                "Sid": "CloudWatchLogsRead",
                "Effect": "Allow",
                "Action": [
                    "logs:GetLogEvents",
                    "logs:FilterLogEvents",
                    "logs:StartQuery",
                    "logs:GetQueryResults",
                    "logs:DescribeLogGroups",
                    "logs:DescribeLogStreams",
                ],
            },
        ],
    }
```

```

        "Resource": [f"arn:aws:logs:{REGION}:{ACCOUNT_ID}:log-group:*"],
    },
    {
        "Sid": "CloudWatchLogsWrite",
        "Effect": "Allow",
        "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
        "Resource": [f"arn:aws:logs:{REGION}:{ACCOUNT_ID}:log-group:/aws/bedrock-agentcore/evaluations/*"],
    },
    {
        "Sid": "CloudWatchMetrics",
        "Effect": "Allow",
        "Action": ["cloudwatch:PutMetricData"],
        "Resource": "*",
    },
    {
        "Sid": "CloudWatchLogsIndexAccess",
        "Effect": "Allow",
        "Action": ["logs:DescribeIndexPolicies", "logs:PutIndexPolicy"],
        "Resource": [
            arn
            for group in LOG_GROUPS
            for arn in (
                f"arn:aws:logs:{REGION}:{ACCOUNT_ID}:log-group:{group}",
                f"arn:aws:logs:{REGION}:{ACCOUNT_ID}:log-group:{group}:*",
            )
        ],
    },
    {
        "Sid": "XRayRead",
        "Effect": "Allow",
        "Action": ["xray:GetTraceSummaries", "xray:BatchGetTraces", "xray:GetTraceGraph"],
        "Resource": "*",
    },
],
}

try:
    role = iam_client.get_role(RoleName=EVALUATION_ROLE_NAME)["Role"]
    print(f"Evaluation role exists: {role['Arn']}")
except ClientError as exc:
    if exc.response["Error"]["Code"] != "NoSuchEntity":
        raise
    role = iam_client.create_role(
        RoleName=EVALUATION_ROLE_NAME,
        AssumeRolePolicyDocument=json.dumps(trust_policy),
        Description="IAM role for AgentCore workshop online evaluation",
    )["Role"]
    print(f"Created evaluation role: {role['Arn']}")
    time.sleep(10)
iam_client.put_role_policy(
    RoleName=EVALUATION_ROLE_NAME,
    PolicyName="evaluation-permissions",
    PolicyDocument=json.dumps(permissions_policy),
)
return role["Arn"]

```

```
EVALUATION_ROLE_ARN = create_or_update_evaluation_role()
print(f"Evaluation role ARN: {EVALUATION_ROLE_ARN}")
```

Created evaluation role: arn:aws:iam::390402578846:role/ecommerce-workshop-evaluation-role
Evaluation role ARN: arn:aws:iam::390402578846:role/ecommerce-workshop-evaluation-role

Step 2b: (Optional) Repair Evaluation Role Log-Group Permissions

Run this cell only if **Step 3** fails with:

ValidationException: The provided execution role does not have permissions to access the specified log groups

This happens when the evaluation execution role is missing CloudWatch Logs index permissions (`logs:PutIndexPolicy` / `logs:DescribeIndexPolicies`) on the log groups referenced by the online evaluation data source (`aws/spans` , the runtime log group, and the vended-logs group). It re-applies the full role policy with index access on every group in `LOG_GROUPS` , then waits for IAM to propagate. Safe to run more than once.

```
In [3]: # Optional recovery cell: repair the evaluation role's Log-group permissions.
import json, time

EVALUATION_ROLE_NAME = "ecommerce-workshop-evaluation-role"

permissions_policy = {
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "BedrockModelAccess",
            "Effect": "Allow",
            "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
            "Resource": [
                f"arn:aws:bedrock:{REGION}::foundation-model/*",
                f"arn:aws:bedrock:*:{ACCOUNT_ID}:inference-profile/*",
            ],
        },
        {
            "Sid": "CloudWatchLogsRead",
            "Effect": "Allow",
            "Action": [
                "logs:GetLogEvents", "logs:FilterLogEvents", "logs:StartQuery",
                "logs:GetQueryResults", "logs:DescribeLogGroups", "logs:DescribeLogStreams",
            ],
            "Resource": ["*"],
        },
        {
            "Sid": "CloudWatchLogsWrite",
            "Effect": "Allow",
            "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "logs:PutLogEvents"],
            "Resource": [f"arn:aws:logs:{REGION}:{ACCOUNT_ID}:log-group:/aws/bedrock-agentcore/evaluations/*"],
        },
        {
            "Sid": "CloudWatchMetrics",
            "Effect": "Allow",
            "Action": ["cloudwatch:PutMetricData"],
            "Resource": "*",
        }
    ]
}
```

```

    },
    {
        # Index permissions on EVERY log group in the online-eval data source.
        "Sid": "CloudWatchLogsIndexAccess",
        "Effect": "Allow",
        "Action": ["logs:DescribeIndexPolicies", "logs:PutIndexPolicy"],
        "Resource": [
            arn
            for group in LOG_GROUPS
            for arn in (
                f"arn:aws:logs:{REGION}:{ACCOUNT_ID}:log-group:{group}",
                f"arn:aws:logs:{REGION}:{ACCOUNT_ID}:log-group:{group}:*"
            )
        ],
    },
    {
        "Sid": "XRayRead",
        "Effect": "Allow",
        "Action": ["xray:GetTraceSummaries", "xray:BatchGetTraces", "xray:GetTraceGraph"],
        "Resource": "*",
    },
],
}

iam_client.put_role_policy(
    RoleName=EVALUATION_ROLE_NAME,
    PolicyName="evaluation-permissions",
    PolicyDocument=json.dumps(permissions_policy),
)
print("Updated evaluation role. Index access granted on:")
for _g in LOG_GROUPS:
    print("  -", _g)
print("Waiting 30s for IAM propagation...")
time.sleep(30)
print("Done. Now (re-)run the Step 3 cell.")

```

Updated evaluation role. Index access granted on:

- aws/spans
- /aws/bedrock-agentcore/runtimes/ecommerce_workshop_product_catalog_agent-518f2MA0eC-DEFAULT
- /aws/vendedlogs/bedrock-agentcore/ecommerce_workshop_product_catalog_agent-518f2MA0eC

Waiting 30s for IAM propagation...

Done. Now (re-)run the Step 3 cell.

Step 3: Create The Online Evaluation Evidence Plane

This cell creates an AgentCore online evaluation configuration for the deployed runtime.

The key concept is that online evaluation is a continuously running evidence collector. It watches CloudWatch logs for sessions from the selected service name, samples those sessions, and applies built-in evaluators that do not require per-session ground truth. The output should show the online evaluation config ID, ARN, status, evaluator list, and sampling rule.

```

In [4]: ONLINE_EVAL_CONFIG_NAME = f"ecommerce_online_evidence_{DEPLOYMENT_MANIFEST['deployment_id'].split('-')[-1]}"
        ONLINE_EVALUATORS = [
            "Builtin.Helpfulness",
            "Builtin.GoalSuccessRate",

```

```

        "Builtin.ToolSelectionAccuracy",
        "Builtin.Coherence",
    ]

def list_online_configs():
    configs = []
    next_token = None
    while True:
        params = {"maxResults": 50}
        if next_token:
            params["nextToken"] = next_token
        response = agentcore_control_client.list_online_evaluation_configs(**params)
        configs.extend(response.get("items", response.get("onlineEvaluationConfigs", [])))
        next_token = response.get("nextToken")
        if not next_token:
            break
    return configs

def online_config_alignment(config):
    cloudwatch = (config.get("dataSourceConfig") or {}).get("cloudWatchLogs") or {}
    configured_logs = list(dict.fromkeys(cloudwatch.get("logGroupNames") or []))
    configured_services = list(dict.fromkeys(cloudwatch.get("serviceNames") or []))
    missing_logs = [group for group in LOG_GROUPS if group not in configured_logs]
    missing_services = [name for name in SERVICE_NAMES if name not in configured_services]
    return {
        "configured_log_group_names": configured_logs,
        "configured_service_names": configured_services,
        "missing_log_group_names": missing_logs,
        "missing_service_names": missing_services,
        "aligned": not missing_logs and not missing_services,
    }

def update_online_config_data_source(details, alignment):
    config_id = details["onlineEvaluationConfigId"]
    merged_logs = list(dict.fromkeys(alignment["configured_log_group_names"] + LOG_GROUPS))
    target_services = SERVICE_NAMES[-1:] or alignment["configured_service_names"][:1]
    params = {
        "onlineEvaluationConfigId": config_id,
        "dataSourceConfig": {
            "cloudWatchLogs": {
                "logGroupNames": merged_logs,
                "serviceNames": target_services,
            }
        },
        "clientToken": str(uuid.uuid4()),
    }
    for key in [
        "description",
        "rule",
        "evaluators",
        "insights",
        "clusteringConfig",
        "evaluationExecutionRoleArn",
    ]:

```

```

        "executionStatus",
    ]:
        value = details.get(key)
        if value is not None:
            params[key] = value
    agentcore_control_client.update_online_evaluation_config(**params)
    updated = agentcore_control_client.get_online_evaluation_config(
        onlineEvaluationConfigId=config_id
    )
    print(
        "Updated online evaluation config data source: "
        f"{config_id} services={target_services}"
    )
    return updated

def get_or_create_online_config():
    for config in list_online_configs():
        if config.get("onlineEvaluationConfigName") == ONLINE_EVAL_CONFIG_NAME:
            config_id = config.get("onlineEvaluationConfigId")
            details = agentcore_control_client.get_online_evaluation_config(
                onlineEvaluationConfigId=config_id
            )
            alignment = online_config_alignment(details)
            if not alignment["aligned"]:
                details = update_online_config_data_source(details, alignment)
            else:
                print(f"Using existing online evaluation config: {config_id}")
            return details
    create_kwargs = dict(
        onlineEvaluationConfigName=ONLINE_EVAL_CONFIG_NAME,
        description=f"Section 04 online evidence for {DEPLOYMENT_MANIFEST['deployment_id']}",
        rule={
            "samplingConfig": {"samplingPercentage": 100.0},
            "sessionConfig": {"sessionTimeoutMinutes": 2},
        },
        dataSourceConfig={
            "cloudWatchLogs": {
                "logGroupNames": LOG_GROUPS,
                "serviceNames": SERVICE_NAMES,
            }
        },
        evaluators=[{"evaluatorId": evaluator_id} for evaluator_id in ONLINE_EVALUATORS],
        evaluationExecutionRoleArn=EVALUATION_ROLE_ARN,
        enableOnCreate=True,
        tags={
            "workshop": "agentic-ai-evaluation-observability",
            "section": "04",
            "deployment_id": DEPLOYMENT_MANIFEST["deployment_id"],
        },
    )
    # The evaluation execution role is created/updated in the previous cell.
    # CreateOnlineEvaluationConfig validates that the role can access the
    # specified log groups, and IAM changes take time to propagate, so a fresh
    # or just-updated role can transiently fail with:
    # "The provided execution role does not have permissions to access the

```

```

# specified log groups"
# Retry with backoff so creation succeeds once IAM catches up.
max_attempts = 6
response = None
for attempt in range(1, max_attempts + 1):
    try:
        response = agentcore_control_client.create_online_evaluation_config(**create_kwargs)
        break
    except ClientError as exc:
        code = exc.response.get("Error", {}).get("Code")
        message = exc.response.get("Error", {}).get("Message", "")
        retrieable = code == "ValidationException" and "does not have permissions" in message
        if not retrieable or attempt == max_attempts:
            raise
        wait_seconds = min(15 * attempt, 60)
        print(
            f" Execution-role permissions not propagated yet "
            f"(attempt {attempt}/{max_attempts}); waiting {wait_seconds}s..."
        )
        time.sleep(wait_seconds)
print(f"Created online evaluation config: {response.get('onlineEvaluationConfigId')}")
return response

ONLINE_EVAL_CONFIG = get_or_create_online_config()
ONLINE_EVAL_CONFIG_ID = ONLINE_EVAL_CONFIG.get("onlineEvaluationConfigId")
ONLINE_EVAL_CONFIG_ARN = ONLINE_EVAL_CONFIG.get("onlineEvaluationConfigArn")
ONLINE_CONFIG_ALIGNMENT = online_config_alignment(ONLINE_EVAL_CONFIG)
print(json.dumps({
    "config_id": ONLINE_EVAL_CONFIG_ID,
    "config_arn": ONLINE_EVAL_CONFIG_ARN,
    "execution_status": ONLINE_EVAL_CONFIG.get("executionStatus"),
    "evaluators": ONLINE_EVALUATORS,
    "service_names": ONLINE_CONFIG_ALIGNMENT["configured_service_names"],
    "missing_service_names": ONLINE_CONFIG_ALIGNMENT["missing_service_names"],
}, indent=2, default=str))

```

Created online evaluation config: ecommerce_online_evidence_51ad2cf5-1BkukcB74g

```

{
  "config_id": "ecommerce_online_evidence_51ad2cf5-1BkukcB74g",
  "config_arn": "arn:aws:bedrock-agentcore:us-east-1:390402578846:online-evaluation-config/ecommerce_online_evidence_51ad2cf5-1BkukcB74g",
  "execution_status": "ENABLED",
  "evaluators": [
    "Builtin.Helpfulness",
    "Builtin.GoalSuccessRate",
    "Builtin.ToolSelectionAccuracy",
    "Builtin.Coherence"
  ],
  "service_names": [],
  "missing_service_names": [
    "ecommerce_workshop_product_catalog_agent.DEFAULT"
  ]
}

```

Step 4: Generate Monitored Workshop Traffic

This cell sends a small set of safe, synthetic sessions through the deployed runtime.

The goal is to create realistic trace material for the feedback loop: recommendation behavior, comparison behavior, RBAC denial behavior, out-of-scope behavior, and an admin read path. Each invocation gets a trace-friendly session ID so later cells can join runtime responses, tool calls, and CloudWatch spans without putting tokens or answer keys into trace attributes. The ID uses hyphenated scenario labels because downstream trace and evaluation services need to correlate the same session across runtime, vended, and span logs.

When the cell finishes, the important output is the list of session IDs, status values, and observed tool usage.

```
In [5]: SUFFIX = demo_user_suffix(DEPLOYMENT_MANIFEST)
USER_POOL_ID = DEPLOYMENT_MANIFEST["cognito"]["user_pool_id"]
USER_CLIENT_ID = DEPLOYMENT_MANIFEST["cognito"]["user_client_id"]
TEST_PASSWORD = os.environ.get("SECTION03_TEST_PASSWORD", f"{SUFFIX}Aa1!z9")
CUSTOMER_EMAIL = os.environ.get("SECTION03_CUSTOMER_EMAIL", f"customer+{SUFFIX}@example.com")
ADMIN_EMAIL = os.environ.get("SECTION03_ADMIN_EMAIL", f"admin+{SUFFIX}@example.com")

customer_tokens = get_user_token(cognito_client, USER_POOL_ID, USER_CLIENT_ID, CUSTOMER_EMAIL, TEST_PASSWORD)
admin_tokens = get_user_token(cognito_client, USER_POOL_ID, USER_CLIENT_ID, ADMIN_EMAIL, TEST_PASSWORD)
if not customer_tokens.get("id_token") or not admin_tokens.get("id_token"):
    raise RuntimeError("Could not authenticate Section 03 demo users for monitored traffic")
print(f"Authenticated customer: {redact_email(CUSTOMER_EMAIL)}")
print(f"Authenticated admin: {redact_email(ADMIN_EMAIL)}")

traffic_scenarios = [
    {
        "actor_role": "customer",
        "scenario_role": "recommendation_gap",
        "prompt": "I bought wireless headphones. Recommend compatible accessories under $80.",
    },
    {
        "actor_role": "customer",
        "scenario_role": "comparison_gap",
        "prompt": "Compare PROD-001 and PROD-008 for battery life and portability.",
    },
    {
        "actor_role": "customer",
        "scenario_role": "rbac_regression",
        "prompt": "Delete product PROD-001 from the catalog.",
    },
    {
        "actor_role": "customer",
        "scenario_role": "out_of_scope",
        "prompt": "Where is my order number 12345?",
    },
    {
        "actor_role": "admin",
        "scenario_role": "admin_observability",
        "prompt": "Check inventory for PROD-001 and summarize the current stock status.",
    },
]

role_tokens = {"customer": customer_tokens, "admin": admin_tokens}
monitored_sessions = []
print("Invoking deployed runtime for monitored traffic...")
for index, scenario in enumerate(traffic_scenarios, start=1):
```

```

scenario_slug = str(scenario["scenario_role"]).replace("_", "-")
session_id = f"s04-{scenario_slug}-{uuid.uuid4().hex}"
tokens = role_tokens[scenario["actor_role"]]
payload = {
    "prompt": scenario["prompt"],
    "bearer_token": tokens.get("id_token", ""),
    "access_token": tokens.get("access_token", ""),
    "session_id": session_id,
}
result = invoke_agent_runtime(agentcore_runtime_client, RUNTIME_ARN, session_id, payload)
item = {
    "session_id": session_id,
    "actor_role": scenario["actor_role"],
    "scenario_role": scenario["scenario_role"],
    "prompt": scenario["prompt"],
    "status": result.get("status", "error"),
    "response": result.get("response", result.get("error", "")),
    "tools_used": result.get("metadata", {}).get("tools_used", []),
    "deployment_id": DEPLOYMENT_MANIFEST["deployment_id"],
    "dataset_baseline_version": BASELINE_DATASET_VERSION,
}
monitored_sessions.append(item)
print(f" [{index}] {item['status']}: {session_id} role={item['actor_role']} tools={item['tools_used']}")
time.sleep(1)

failures = [item for item in monitored_sessions if item["status"] != "success"]
if failures:
    print(f"Warning: {len(failures)} monitored invocations returned non-success status")

```

Got tokens for user: cu***@example.com

Got tokens for user: ad***@example.com

Authenticated customer: cu***@example.com

Authenticated admin: ad***@example.com

Invoking deployed runtime for monitored traffic...

```

[1] success: s04-recommendation-gap-53437f3eea374610bed65267c9debb32 role=customer tools=['get_product_recommendations', 'get_product_recommendations']
[2] success: s04-comparison-gap-a5d046dcf082446997227717a0b2bda9 role=customer tools=['compare_products']
[3] success: s04-rbac-regression-31190179199d4b26b7cefd753e310d57 role=customer tools=[]
[4] success: s04-out-of-scope-4846131442f449bb923b605403a04b08 role=customer tools=[]
[5] success: s04-admin-observability-fc3d7e71ad284e43b8bb60788b89cb64 role=admin tools=['check_inventory']

```

Step 5: Collect Traces And Online Signals

This cell reads CloudWatch events for the monitored session IDs and reconstructs the agent execution trail.

The key concept is correlation. The runtime response tells you what the user saw. The spans and AgentCore runtime trace records tell you what the agent and tools did. The online evaluation metrics tell you whether automated scoring has started to land. These signals arrive asynchronously, so the notebook collects what is available and records the status clearly.

The output should help you answer three questions: did trace records arrive, which custom spans or runtime operations were found, and which tools were observed per session?

```

In [6]: def parse_otlp_attributes(attrs):
        if isinstance(attrs, dict):
            return attrs
        parsed = {}
        for item in attrs or []:

```

```

        key = item.get("key")
        value = item.get("value", {})
        if not key:
            continue
        if isinstance(value, dict):
            parsed[key] = next(iter(value.values()), None)
        else:
            parsed[key] = value
    return parsed

def extract_session_id_from_body(body):
    text = str(body or "")
    match = re.search(r"session_id=([A-Za-z0-9_.-]+)", text)
    return match.group(1) if match else None

def span_documents_from_message(message):
    records = []
    try:
        data = json.loads(message)
    except json.JSONDecodeError:
        return records

    def add_record(record, *, fallback_name="runtime.trace_record"):
        if not isinstance(record, dict):
            return
        attrs = parse_otlp_attributes(record.get("attributes") or {})
        resource_attrs = parse_otlp_attributes((record.get("resource") or {}).get("attributes", {}))
        for key, value in resource_attrs.items():
            attrs.setdefault(key, value)
        session_id = record.get("session_id") or attrs.get("session.id") or attrs.get("runtime.session_id")
        if not session_id:
            session_id = extract_session_id_from_body(record.get("body"))
        if session_id:
            attrs.setdefault("session.id", session_id)
            attrs.setdefault("runtime.session_id", session_id)
        name = record.get("name") or attrs.get("event.name") or record.get("operation") or fallback_name
        if record.get("traceId") or record.get("trace_id") or record.get("spanId") or record.get("span_id") or session_id:
            records.append(
                {
                    "name": name,
                    "attributes": attrs,
                    "traceId": record.get("traceId") or record.get("trace_id"),
                    "spanId": record.get("spanId") or record.get("span_id"),
                    "_record_type": "trace_record",
                }
            )

    if isinstance(data, dict) and data.get("name") and data.get("attributes"):
        add_record(data, fallback_name=data.get("name"))
    elif isinstance(data, dict):
        add_record(data)

    for resource_span in data.get("resourceSpans", []) if isinstance(data, dict) else []:
        resource_attrs = parse_otlp_attributes(resource_span.get("resource", {}).get("attributes", []))

```

```

        for scope_span in resource_span.get("scopeSpans", []):
            for span in scope_span.get("spans", []):
                doc = dict(span)
                attrs = parse_otlp_attributes(doc.get("attributes", []))
                attrs.update({k: v for k, v in resource_attrs.items() if k not in attrs})
                doc["attributes"] = attrs
                add_record(doc, fallback_name=doc.get("name") or "runtime.trace_record")
    return records

def collect_session_spans(session_ids, *, timeout_seconds=None, poll_seconds=15):
    timeout_seconds = int(os.environ.get("SECTION04_TRACE_TIMEOUT_SECONDS", timeout_seconds or 360))
    deadline = time.time() + timeout_seconds
    session_ids = list(session_ids)
    collected = []
    seen_event_ids = set()
    while time.time() < deadline:
        for log_group in LOG_GROUPS:
            for session_id in session_ids:
                try:
                    response = logs_client.filter_log_events(
                        logGroupName=log_group,
                        filterPattern=f'"{session_id}"',
                        startTime=int((time.time() - 3600) * 1000),
                        endTime=int((time.time() + 60) * 1000),
                        limit=100,
                    )

                except Exception as exc:
                    print(f" Trace query warning for {log_group}: {sanitize_error(exc)}")
                    continue
                for event in response.get("events", []):
                    event_id = event.get("eventId") or f"{log_group}:{event.get('timestamp')}:{hash(event.get('message', ''))}"
                    if event_id in seen_event_ids:
                        continue
                    seen_event_ids.add(event_id)
                    spans = span_documents_from_message(event.get("message", ""))
                    for span in spans:
                        span.setdefault("_log_group", log_group)
                        collected.append(span)

    if collected:
        sessions_with_spans = {
            (span.get("attributes") or {}).get("session.id")
            or (span.get("attributes") or {}).get("runtime.session_id")
            for span in collected
        }
        if any(session_id in sessions_with_spans for session_id in session_ids):
            break
    print(f" Waiting {poll_seconds}s for CloudWatch trace propagation...")
    time.sleep(poll_seconds)
    return collected

print("Collecting CloudWatch spans for monitored sessions...")
recent_spans = collect_session_spans([item["session_id"] for item in monitored_sessions])
session_tools = extract_tool_calls_from_spans(recent_spans)
for item in monitored_sessions:
    item["tools_used_from_spans"] = session_tools.get(item["session_id"], [])

```

```

trace_ids = sorted({span.get("traceId") or span.get("trace_id") for span in recent_spans if span.get("traceId") or span.get("trace_id")})
trace_summary = {
    "span_count": len(recent_spans),
    "trace_count": len(trace_ids),
    "session_count_with_tools": len(session_tools),
    "custom_span_names_found": sorted({span.get("name") for span in recent_spans if str(span.get("name", "")).startswith("product_catalog.')})),
}
print(json.dumps(trace_summary, indent=2))

metric_summary = {"status": "PENDING", "datapoints": []}
try:
    for metric_name in ["Builtin.GoalSuccessRate", "Builtin.Helpfulness", "Builtin.ToolSelectionAccuracy", "Builtin.Coherence"]:
        response = cloudwatch_client.get_metric_statistics(
            Namespace="Bedrock-AgentCore/Evaluations",
            MetricName=metric_name,
            Dimensions=[{"Name": "service.name", "Value": OTEL_SERVICE_NAME}],
            StartTime=datetime.now(timezone.utc) - timedelta(hours=3),
            EndTime=datetime.now(timezone.utc),
            Period=900,
            Statistics=["Average", "SampleCount"],
        )
        datapoints = response.get("Datapoints", [])
        if datapoints:
            metric_summary["datapoints"].append({
                "metric_name": metric_name,
                "count": len(datapoints),
                "latest_average": sorted(datapoints, key=lambda item: item["Timestamp"])[-1].get("Average"),
            })
        if metric_summary["datapoints"]:
            metric_summary["status"] = "AVAILABLE"
except Exception as exc:
    metric_summary = {"status": "ERROR", "error": sanitize_error(exc)}
print(json.dumps(metric_summary, indent=2, default=str))

```

Collecting CloudWatch spans for monitored sessions...

Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...
Waiting 15s for CloudWatch trace propagation...

```
{
  "span_count": 0,
  "trace_count": 0,
  "session_count_with_tools": 0,
  "custom_span_names_found": []
}
{
  "status": "AVAILABLE",
  "datapoints": [
    {
      "metric_name": "Builtin.GoalSuccessRate",
      "count": 2,
      "latest_average": 1.0
    },
    {
      "metric_name": "Builtin.Helpfulness",
      "count": 2,
      "latest_average": 0.916
    }
  ]
}
```

Step 6: Create The Evidence Dashboard And Manifest

This cell packages the observable state into a dashboard and a manifest.

The dashboard is for humans who want to inspect runtime and evaluation signals in CloudWatch. The manifest is for later notebooks and automation. It records the online evaluation config, dashboard URL, log groups, service names, monitored sessions, trace summary, metric status, and dataset lineage.

The thing to learn here is the separation between visual inspection and machine-readable lineage: both point to the same evidence, but they serve different readers.

```
In [7]: DASHBOARD_NAME = f"EcommerceWorkshop-OnlineEvidence-{DEPLOYMENT_MANIFEST['deployment_id'].split('-')[-1]}"
        RUNTIME_DIMENSION_NAME = f"{DEPLOYMENT_MANIFEST['runtime']['runtime_name']}:DEFAULT"
        EVALUATION_METRIC_SERVICE_NAME = SERVICE_NAMES[-1] if SERVICE_NAMES else OTEL_SERVICE_NAME
```

```

def runtime_metric(metric_name, stat, label=None):
    entry = [
        "AWS/Bedrock-AgentCore",
        metric_name,
        "Resource", RUNTIME_ARN,
        "Operation", "InvokeAgentRuntime",
        "Name", RUNTIME_DIMENSION_NAME,
        {"stat": stat},
    ]
    if label:
        entry[-1]["label"] = label
    return entry

widgets = [
    {
        "type": "metric", "x": 0, "y": 0, "width": 12, "height": 6,
        "properties": {
            "title": "Runtime Invocations",
            "metrics": [runtime_metric("Invocations", "Sum")],
            "region": REGION,
            "period": 300,
            "view": "timeSeries",
        },
    },
    {
        "type": "metric", "x": 12, "y": 0, "width": 12, "height": 6,
        "properties": {
            "title": "Online Evaluation Signals",
            "metrics": [
                ["Bedrock-AgentCore/Evaluations", "Builtin.GoalSuccessRate", "service.name", EVALUATION_METRIC_SERVICE_NAME, {"stat": "Average"}],
                [".", "Builtin.Helpfulness", ".", ".", {"stat": "Average"}],
                [".", "Builtin.ToolSelectionAccuracy", ".", ".", {"stat": "Average"}],
            ],
            "region": REGION,
            "period": 900,
            "view": "timeSeries",
        },
    },
    {
        "type": "log", "x": 0, "y": 6, "width": 24, "height": 6,
        "properties": {
            "title": "Recent Product Catalog Spans",
            "query": (
                "SOURCE 'aws/spans'\n"
                "| filter @message like /product_catalog/\n"
                "| fields @timestamp, @message\n"
                "| sort @timestamp desc\n"
                "| limit 50"
            ),
            "region": REGION,
            "view": "table",
        },
    },
]

dashboard_body = json.dumps({"widgets": widgets})

```

```

dashboard_status = "SKIPPED"
try:
    response = cloudwatch_client.put_dashboard(DashboardName=DASHBOARD_NAME, DashboardBody=dashboard_body)
    dashboard_status = "DEPLOYED_WITH_WARNINGS" if response.get("DashboardValidationMessages") else "DEPLOYED"
except Exception as exc:
    dashboard_status = "ERROR"
    print(f"Dashboard deployment warning: {sanitize_error(exc)}")

dashboard = {
    "dashboard_name": DASHBOARD_NAME,
    "status": dashboard_status,
    "url": f"https://{REGION}.console.aws.amazon.com/cloudwatch/home?region={REGION}#dashboards/dashboard/{DASHBOARD_NAME}",
}

query_templates = {
    "recent_spans": "SOURCE 'aws/spans' | filter @message like /product_catalog/ | sort @timestamp desc | limit 50",
    "session_lookup": "SOURCE 'aws/spans' | filter @message like /<session-id>/ | sort @timestamp desc | limit 50",
}

online_config_summary = {
    "online_evaluation_config_name": ONLINE_EVAL_CONFIG_NAME,
    "online_evaluation_config_id": ONLINE_EVAL_CONFIG_ID,
    "online_evaluation_config_arn": ONLINE_EVAL_CONFIG_ARN,
    "execution_status": ONLINE_EVAL_CONFIG.get("executionStatus"),
    "status": ONLINE_EVAL_CONFIG.get("status"),
    "evaluator_ids": ONLINE_EVALUATORS,
    "sampling_percentage": 100.0,
    "evaluation_execution_role_arn": EVALUATION_ROLE_ARN,
}

online_evidence_manifest = build_online_evidence_manifest(
    deployment=DEPLOYMENT_MANIFEST,
    dataset=DATASET_MANIFEST,
    batch_evaluation=BATCH_EVALUATION_MANIFEST,
    online_config=online_config_summary,
    dashboard=dashboard,
    monitored_sessions=[
        {k: v for k, v in item.items() if k not in {"response"}}
        for item in monitored_sessions
    ],
    trace_summary=trace_summary,
    metric_summary=metric_summary,
    query_templates=query_templates,
)

save_json(online_evidence_manifest, ONLINE_EVIDENCE_MANIFEST_PATH)
print(f"Online evidence manifest saved: {ONLINE_EVIDENCE_MANIFEST_PATH}")
print(json.dumps({"dashboard": dashboard, "trace_summary": trace_summary, "metric_status": metric_summary['status']}, indent=2, default=str))

```

Online evidence manifest saved: /workshop/04-online-eval-observability/online_evidence_manifest.json

```
{
  "dashboard": {
    "dashboard_name": "EcommerceWorkshop-OnlineEvidence-51ad2cf5",
    "status": "DEPLOYED",
    "url": "https://us-east-1.console.aws.amazon.com/cloudwatch/home?region=us-east-1#dashboards/dashboard/EcommerceWorkshop-OnlineEvidence-51ad2cf5"
  },
  "trace_summary": {
    "span_count": 0,
    "trace_count": 0,
    "session_count_with_tools": 0,
    "custom_span_names_found": []
  },
  "metric_status": "AVAILABLE"
}
```

Step 7: Mine And Review Production Feedback Candidates

This cell has two phases.

First, the mining phase scans the monitored sessions and creates a review queue. A session becomes a candidate when it has a useful signal: a recommendation gap, comparison gap, RBAC/security behavior, weak response wording, or a runtime failure. The candidate records the sanitized user input, sanitized agent output, observed tools, priority, and reason.

Second, the review phase chooses which candidates are safe and useful enough to become dataset examples. Review is where a candidate receives an expected response, expected tool trajectory, assertions, reviewer metadata, and a scenario ID. Runtime invocation failures stay in the review queue as investigation signals; they are not promoted as expected responses.

After running the cell, inspect the two tables. The first table is the mined review queue. The second table is the smaller set of reviewed examples that will be sent to the managed dataset in the next step.

```
In [8]: feedback_candidates = build_feedback_candidates(
        deployment=DEPLOYMENT_MANIFEST,
        dataset=DATASET_MANIFEST,
        monitored_sessions=monitored_sessions,
        session_tools=session_tools,
    )
    save_json(feedback_candidates, PRODUCTION_FEEDBACK_CANDIDATES_PATH)

    candidate_rows = []
    for candidate in feedback_candidates["candidates"]:
        candidate_rows.append(
            {
                "candidate_id": candidate["candidate_id"],
                "scenario_role": candidate["scenario_role"],
                "actor_role": candidate["actor_role"],
                "priority": candidate["signal_summary"]["priority"],
                "reason": candidate["signal_summary"]["reason"],
                "observed_tools": ", ".join(candidate.get("actual_tools") or []),
                "promote_to_dataset?": candidate["recommended_for_promotion"],
                "runtime/canary investigation?": candidate.get("recommended_for_canary_investigation", False),
            }
        )
```

```

print(f"Feedback candidates saved: {PRODUCTION_FEEDBACK_CANDIDATES_PATH}")
print(f"Candidate count: {feedback_candidates['candidate_count']}")
display(pd.DataFrame(candidate_rows))

promoted_feedback_examples = promote_feedback_candidates(
    feedback_candidates,
    reviewer="section04_workshop_review",
    max_examples=2,
)
save_json(promoted_feedback_examples, PROMOTED_FEEDBACK_EXAMPLES_PATH)

review_rows = []
for decision, example in zip(
    promoted_feedback_examples["review_decisions"],
    promoted_feedback_examples["managed_dataset_examples"],
):
    review_rows.append(
        {
            "scenario_id": decision["scenario_id"],
            "source_candidate_id": decision["candidate_id"],
            "source_session_id": decision["source_session_id"],
            "category": example.get("metadata", {}).get("category"),
            "expected_tools": ", ".join(
                example.get("expected_trajectory", {}).get("toolNames", [])
            ),
            "decision": decision["decision"],
        }
    )

print(f"Promoted examples saved: {PROMOTED_FEEDBACK_EXAMPLES_PATH}")
print(f"Promoted count: {promoted_feedback_examples['promoted_count']}")
display(pd.DataFrame(review_rows))

```

Feedback candidates saved: /workshop/04-online-eval-observability/production_feedback_candidates.json

Candidate count: 4

	candidate_id	scenario_role	actor_role	priority	reason	observed_tools	promote_to_dataset?	runtime/canary investigation?
0	feedback-5267c9debb32	recommendation_gap	customer	medium	recommendation_gap	get_product_recommendations, get_product_recom...	True	False
1	feedback-7717a0b2bda9	comparison_gap	customer	medium	comparison_gap	compare_products	True	False
2	feedback-fd753e310d57	rbac_regression	customer	medium	rbac_regression		True	False
3	feedback-605403a04b08	out_of_scope	customer	medium	weak_response_signal		True	False

Promoted examples saved: /workshop/04-online-eval-observability/promoted_feedback_examples.json

Promoted count: 2

	scenario_id	source_candidate_id	source_session_id	category	expected_tools	decision
0	prod-feedback-5267c9debb32	feedback-5267c9debb32	s04-recommendation-gap-53437f3eea374610bed6526...	recommendation_gap	get_product_recommendations, get_product_recom...	promote_to_dataset_draft
1	prod-feedback-7717a0b2bda9	feedback-7717a0b2bda9	s04-comparison-gap-a5d046dcf082446997227717a0b...	comparison_gap	compare_products	promote_to_dataset_draft

Step 8: Publish Reviewed Examples To The Managed Dataset

This cell takes the reviewed examples from Step 7 and adds them to the existing AgentCore managed dataset draft.

Before writing anything, it checks whether the scenario IDs already exist so reruns do not duplicate examples. If there are new examples, it adds them to the draft and publishes the next immutable dataset version. The manifest records the previous version, updated version, added scenario IDs, and source sessions.

The important thing to look for is the update status: `UPDATED_AND_PUBLISHED` means a new version was created; `NOOP_ALREADY_PRESENT` means the reviewed examples were already in the draft.

```
In [9]: from bedrock_agentcore.evaluation import DatasetClient

def list_all_dataset_examples(dataset_id):
    examples = []
    next_token = None
    while True:
        params = {"datasetId": dataset_id}
        if next_token:
            params["nextToken"] = next_token
        response = agentcore_control_client.list_dataset_examples(**params)
        examples.extend(response.get("examples", []))
        next_token = response.get("nextToken")
        if not next_token:
            break
    return examples

def list_all_dataset_versions(dataset_id):
    versions = []
    next_token = None
    while True:
        params = {"datasetId": dataset_id}
        if next_token:
            params["nextToken"] = next_token
        response = agentcore_control_client.list_dataset_versions(**params)
        versions.extend(response.get("versions", []))
        next_token = response.get("nextToken")
        if not next_token:
            break
    return versions

prior_versions = list_all_dataset_versions(DATASET_ID)
existing_examples = list_all_dataset_examples(DATASET_ID)
```

```

existing_scenario_ids = {example.get("scenario_id") for example in existing_examples}
examples_to_add = [
    example for example in promoted_feedback_examples["managed_dataset_examples"]
    if example.get("scenario_id") not in existing_scenario_ids
]

print(f"Existing draft examples: {len(existing_examples)}")
print(f"Examples selected for add: {len(examples_to_add)}")

dataset_client = DatasetClient(region_name=REGION)
add_response = None
publish_response = None
if examples_to_add:
    add_response = dataset_client.add_examples_and_wait(
        datasetId=DATASET_ID,
        source={"inlineExamples": {"examples": examples_to_add}},
    )
    publish_response = dataset_client.create_dataset_version_and_wait(datasetId=DATASET_ID)
    update_status = "UPDATED_AND_PUBLISHED"
else:
    update_status = "NOOP_ALREADY_PRESENT"

new_versions = list_all_dataset_versions(DATASET_ID)
dataset_update_manifest = build_dataset_update_manifest(
    deployment=DEPLOYMENT_MANIFEST,
    dataset=DATASET_MANIFEST,
    promoted_examples={
        **promoted_feedback_examples,
        "managed_dataset_examples": examples_to_add or promoted_feedback_examples["managed_dataset_examples"],
    },
    prior_versions=prior_versions,
    new_versions=new_versions,
    add_examples_response=add_response,
    publish_response=publish_response,
    status=update_status,
)
save_json(dataset_update_manifest, DATASET_UPDATE_MANIFEST_PATH)
print(f"Dataset update manifest saved: {DATASET_UPDATE_MANIFEST_PATH}")
print(json.dumps({
    "status": dataset_update_manifest["status"],
    "previous_latest_version": dataset_update_manifest["previous_latest_version"],
    "updated_dataset_version": dataset_update_manifest["updated_dataset_version"],
    "added_example_count": dataset_update_manifest["added_example_count"],
}, indent=2))

```

Existing draft examples: 5

Examples selected for add: 2

Dataset update manifest saved: /workshop/04-online-eval-observability/dataset_update_manifest.json

```

{
  "status": "UPDATED_AND_PUBLISHED",
  "previous_latest_version": "1",
  "updated_dataset_version": "2",
  "added_example_count": 2
}

```

Step 9: Write The Section Summary

This cell writes the timing and artifact summary for the section.

The files listed here are the handoff points for the next part of the lifecycle: online evidence, feedback candidates, reviewed examples, and dataset-update lineage. If you want to understand what changed without rerunning the notebook, these artifacts are the first place to look.

```
In [10]: SECTION04_DURATION_SECONDS = round(time.time() - NOTEBOOK_STARTED_AT, 1)
section04_timing = {
    "notebook": "04-online-evidence-and-feedback-loop.ipynb",
    "executed_at": datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z"),
    "duration_seconds": SECTION04_DURATION_SECONDS,
    "status": "PASSED",
    "artifacts": [
        str(ONLINE_EVIDENCE_MANIFEST_PATH.relative_to(REPO_ROOT)),
        str(PRODUCTION_FEEDBACK_CANDIDATES_PATH.relative_to(REPO_ROOT)),
        str(PROMOTED_FEEDBACK_EXAMPLES_PATH.relative_to(REPO_ROOT)),
        str(DATASET_UPDATE_MANIFEST_PATH.relative_to(REPO_ROOT)),
    ],
}
save_json(section04_timing, SECTION_DIR / "section04_timing.json")

print("=" * 70)
print("SECTION 04 COMPLETE")
print("=" * 70)
print(f"Duration: {SECTION04_DURATION_SECONDS}s")
print(f"Online evidence: {ONLINE_EVIDENCE_MANIFEST_PATH}")
print(f"Feedback candidates: {PRODUCTION_FEEDBACK_CANDIDATES_PATH}")
print(f"Promoted examples: {PROMOTED_FEEDBACK_EXAMPLES_PATH}")
print(f"Dataset update: {DATASET_UPDATE_MANIFEST_PATH}")
print(f"Dataset update status: {dataset_update_manifest['status']}")
print(f"Updated dataset version: {dataset_update_manifest['updated_dataset_version']}")

try:
    get_ipython().run_line_magic("store", "online_evidence_manifest")
    get_ipython().run_line_magic("store", "feedback_candidates")
    get_ipython().run_line_magic("store", "promoted_feedback_examples")
    get_ipython().run_line_magic("store", "dataset_update_manifest")
except Exception:
    pass
```

=====

SECTION 04 COMPLETE

=====

Duration: 484.4s

Online evidence: /workshop/04-online-eval-observability/online_evidence_manifest.json

Feedback candidates: /workshop/04-online-eval-observability/production_feedback_candidates.json

Promoted examples: /workshop/04-online-eval-observability/promoted_feedback_examples.json

Dataset update: /workshop/04-online-eval-observability/dataset_update_manifest.json

Dataset update status: UPDATED_AND_PUBLISHED

Updated dataset version: 2

Stored 'online_evidence_manifest' (dict)

Stored 'feedback_candidates' (dict)

Stored 'promoted_feedback_examples' (dict)

Stored 'dataset_update_manifest' (dict)